OBJECT ORIENTED AND MODEL DESIGNING

Q1) a) Explain elements of a class diagram with examples.

A class diagram is a static structural diagram that describes the structure of a system by showing the system's classes, their attributes, operations, and the relationships among them.

The primary elements are:

1. **Class:** A blueprint for creating objects, represented by a rectangle with three compartments:
   - **Name:** The name of the class (e.g., Car).
   - **Attributes:** The properties or data the class holds (e.g., color: String).
   - **Operations:** The actions or methods the class can perform (e.g., startEngine()).
2. **Relationships:** Lines that connect classes to show how they interact. The main types are Association, Generalization, and Dependency.

```
+-----------------+
|       Car       |
+-----------------+
| - color: String |
| - speed: float  |
+-----------------+
| + startEngine() |
| + accelerate()  |
+-----------------+
```

Q1) b) What do you mean by Association Class?

An **Association Class** is a class attached to an association relationship between two other classes. It is used when you need to store information about the relationship itself. This is necessary when attributes belong to the link between objects, not to either of the objects themselves.

Example: The Enrollment class below is an association class. The grade of a student belongs to neither the Student nor the Course alone, but to the act of enrollment.

```
+-----------+                              +----------+
|  Student  | 1..* -------- 1..* |  Course  |
+-----------+                              +----------+
      |
      | (dashed line)
      |
+------------+
| Enrollment |
+------------+
| - grade    |
+------------+
```

Q1) c) Explain different Relations in UML.

1. **Dependency:** A "using" relationship where a change in one class (supplier) may affect another (client). It is shown as a dashed arrow. [Class A] - - - > [Class B]
2. **Association:** A structural "has-a" relationship showing that objects of one class are connected to objects of another. It is shown as a solid line. [Class A] ----------------[Class B]
3. **Generalization (Inheritance):** An "is-a" relationship between a general class (superclass) and a specific class (subclass). It is shown as a solid line with a hollow arrowhead pointing to the superclass. [Subclass] |> [Superclass]
4. **Realization:** A relationship where one element (like a class) implements the behavior specified by another (like an interface). It is shown as a dashed line with a hollow arrowhead. [Class] - - - -|> [Interface]

Q2) a) What are different types of analysis classes?

Analysis classes are stereotypes used to model a system's functionality. The three main types are:

1. **Boundary Class:** Represents the interaction between the system and its users or other systems (e.g., user interfaces).
   - **Example:** A LoginScreen class.
2. **Control Class:** Manages the business logic and flow of events, coordinating between boundary and entity classes.
   - **Example:** A LoginController class.
3. **Entity Class:** Represents the persistent data or information of the system (e.g., database tables).
   - **Example:** A User class holding username and password.

Q2) b) How you model class & object in UML?

A class is a blueprint, and an object is an instance of that blueprint.

- **Class:** Modeled as a rectangle with compartments for the name, attributes, and operations.
- **Object:** Modeled as a rectangle with the name underlined, showing objectName:ClassName. It can also show specific attribute values.

```
     CLASS (Blueprint)              OBJECT (Instance)
+-------------------------+    +----------------------------+
|           Car           |    |       myCar : Car          |
+-------------------------+    +----------------------------+
| - make: String          |    | - make = "Toyota"          |
| - model: String         |    | - model = "Camry"          |
+-------------------------+    +----------------------------+
| + start()               |
+-------------------------+
```

Q2) c) Draw a class diagram for ATM system.

```
+------------+ 1       * +-----------+
|    Bank    |<>-------|    ATM    |
+------------+         +-----------+
| - bankCode |         | - location|
+------------+         +-----------+
      | 1
      | manages
      | *
+------------+ 1       * +-------------+
|  Account   |<>-------| Transaction |
+------------+         +-------------+
| - accNumber|         | - amount    |
| - balance  |         +-------------+
+------------+
      ^
      | 1
+------------+ 1       1 +-----------+
|  Customer  |<>-------|   Card    |
+------------+         +-----------+
| - name     |         | - cardNum |
+------------+         +-----------+
```

Q3) a) Explain following: i) Composite state, ii) Self transition, iii) Orthogonal states.

i) **Composite State:** A state that is decomposed into its own nested sub-states, hiding complexity. An object can be in the composite state and only one of its sub-states at a time.

Example:

Authentication in ATM system is a composite state containing:

- Enter PIN
- Verify PIN

Notation/ Diagram:

```
+--------------------------+
| Authentication           |  <-- Composite State
| +-------------------+    |
| | Enter PIN         | |  <-- Sub-state
| +-------------------+    |
| | Verify PIN        | |  <-- Sub-state
| +-------------------+    |
+--------------------------+
```

ii) **Self Transition:** A transition where the source and target state are the same. It models an event that triggers an action but doesn't cause a state change.
Example:
In Idle state, pressing Cancel repeatedly → stays in Idle

- **Notation/ Diagram :**

```
+---------+
|  Idle   |
|         |
|  ↺ Event|  <-- Loops back to itself
+---------+
```

iii) **Orthogonal States:** Independent state machines that are active concurrently within the same composite state. The object is in a state from each of these regions simultaneously.
Example:
In Railway Reservation System:
Region 1 → Seat Selection
Region 2 → Payment Processing
Both regions active at the same time inside a composite state Reservation

- **Notation/ Diagram:**

```
+-----------------------------+
|       Reservation           |  <-- Composite State
| +-----------+ +---------+   |
| | Seat      | | Payment |   |  <-- Orthogonal / parallel regions
| | Selection | |Processing|  |
| +-----------+ +---------+   |
+-----------------------------+
```

Q3) c) Explain terms: i) Aggregation ii) Composition.

1) **Aggregation:** A special type of association representing a "part-of" relationship. The "part" can exist independently of the "whole." It's shown with an unfilled diamond.
Notation: A hollow diamond at the end of the association line near the whole class.
Example: A Department and Professor. If the Department closes, the Professors can still exist.
Diagram:-
Department ◆—— Professor
2) **Composition:** A stronger "part-of" relationship where the "part" cannot exist without the "whole." If the whole is destroyed, the parts are destroyed too. It's shown with a filled diamond.
3) **Notation**: A filled diamond at the end of the association line near the whole class.
Example: A House and its Rooms. If the house is demolished, the rooms cease to exist.
Diagram:-
House ◆—— Room

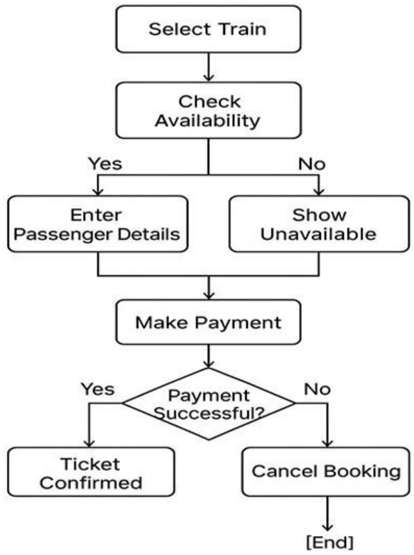
Q4) a) Explain different components of a state diagram.
A state diagram models the dynamic behavior of an object. Its main components are:

1. **State:** A condition in the life of an object. Can be an **Initial State** (starting point), **Final State** (ending point), or a **Simple State**.

2. **Transition:** A change from a source state to a target state, triggered by an event. It's shown as an arrow.
3. **Event:** An occurrence that can trigger a state transition.
4. **Action:** An instantaneous process that occurs when a transition is taken.
5. **Guard Condition:** A boolean expression in [brackets] that must be true for a transition to occur.

Q4) c) Draw state chart diagram for Railway Reservation system.

Definition: A state chart diagram shows different states of an object/system and transitions caused by events. Useful for dynamic behavior modeling of a system.



SOLVED OOMD PAPER - 2023 (P5195)

Q1) a) Explain the types of relationships along with their notations.
Relationships in UML show how model elements are connected. The main types are:

1. **Dependency:** A "using" relationship.
   Notation: A dashed arrow (- - - >).

2. **Association:** A structural "has-a" relationship.
   Notation: A solid line (--------).

3. **Aggregation:** A "part-of" relationship where the part can exist independently.
   Notation: A solid line with an unfilled diamond (<> --------).
4. **Composition:** A strong "part-of" relationship where the part cannot exist without the whole.
   Notation: A solid line with a filled diamond (<#>---------).
5. **Generalization (Inheritance):** An "is-a" relationship.
   Notation: A solid line with a hollow arrowhead (------- |>).

Q1) b) Draw class diagram for ATM system.
(This is a repeated topic)

```
+------------+ 1      * +-----------+
|    Bank    |<>-------|    ATM    |
+------------+         +-----------+
| - bankCode |         | - location|
+------------+         +-----------+
      | 1
      | manages
      | *
+------------+ 1      * +-------------+
|  Account   |<>-------| Transaction |
+------------+         +-------------+
| - accNumber|         | - amount    |
| - balance  |         +-------------+
+------------+
      ^
      | 1
+------------+ 1      1 +-----------+
|  Customer  |<>-------|   Card    |
+------------+         +-----------+
| - name     |         | - cardNum |
+------------+         +-----------+
```

Q1) c) Define object? Explain components of object diagram in brief.

An **object** is an instance of a class. It represents a real-world entity with a state (attributes) and behavior (operations). An **object diagram** shows a snapshot of the system's objects and their relationships (links) at a specific time.

Its components are:

1. **Object:** Represented by a rectangle with its name underlined (objectName : ClassName).
2. **Attribute Values:** Specific values for an object's attributes can be shown (e.g., color = "Blue").
3. **Link:** A connection between two objects, representing an instance of an association.

Q2) a) Difference between link and association with suitable example.

Basis	Association	Link
Level	**Class Level (Blueprint):** Defines a relationship that *can* exist between classes.	**Object Level (Instance):** Represents a connection that *does* exist between objects.
Definition	A semantic relationship between classes that specifies connections among their instances.	A physical or conceptual connection between objects; an instance of an association.
Example	A `Student` class has an association with a `Course` class.	An object `john:Student` has a link to an object `cs101:Course`.

Q2) c) Explain the following terms with respect to object oriented methodology.
i) Abstraction ii) Encapsulation iii) Sharing

i) Abstraction Abstraction is the principle of hiding complex implementation details from the user and showing only the essential, relevant features of an object. It simplifies the interaction with an object by providing a high-level view and reducing complexity. The main goal of abstraction is to manage the complexity of large systems.

- **Example:** When you drive a car, you interact with a simple interface (steering wheel, pedals, gear stick). You don't need to know the complex mechanics of the engine or transmission to operate it. The internal complexity is abstracted away.

ii) Encapsulation Encapsulation is the mechanism of bundling the data (attributes) and the methods (operations) that operate on that data into a single unit, known as a class. It also involves restricting direct access to an object's internal state; this is called data hiding.

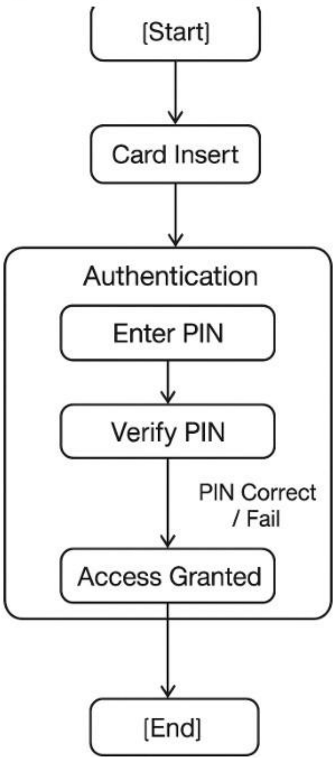
- **Example:** A medical capsule encapsulates the medicine (data) inside a protective shell. The shell acts as an interface, preventing accidental misuse. Similarly, in a class, private attributes are protected, and they can only be changed through public methods, ensuring data integrity.

iii) Sharing (Inheritance) Sharing, in the context of OO methodology, refers to the concept of **inheritance**. It is a mechanism where a new class (a subclass or child class) can derive, or inherit, attributes and methods from an existing class (a superclass or parent class). This promotes code reuse and establishes a clear hierarchical relationship between classes.

- **Example:** Car and Truck classes can both inherit common properties like speed and methods like startEngine() from a more general Vehicle class.

Q3) a) Draw state diagram with composite states from ATM card reading & authentication system.
(This is a repeated topic)

ATM Example – Composite State Diagram



i) Package A package in UML is a grouping mechanism used to organize elements of a model, such as classes, interfaces, and use cases, into related groups. It is visually represented as a file folder symbol. Packages are essential for managing the complexity of large-scale systems by dividing them into smaller, more manageable and understandable subsystems.

ii) Derived Data A derived element, such as a derived attribute or association, is one whose value can be computed or inferred from other information present in the model. It does not represent a new piece of information but rather a value that can be calculated. In a class diagram, a derived attribute is denoted by a forward slash / before its name.

- **Example: In a Person class that has a birthDate attribute, the age can be a derived attribute (/age), as it can always be calculated from the current date and the person's birth date.**

iii) Constraints A constraint is a rule or condition that specifies a restriction on a UML model element. It must always be maintained as true. Constraints are written in plain text and enclosed in curly braces {}. They are used to add rules to the model that cannot be expressed by the standard UML notations.

- **Example: An Account class might have a constraint {balance >= 0.00} on its balance attribute to ensure it never becomes negative.**

Q3) c) Define event and explain in brief different types of events. [5]
An event is a significant or noteworthy occurrence that has a location in time and space. In the context of a UML state machine, an event is a stimulus that can trigger an object to transition from one state to another.

The four main types of events in UML are:

1. **Signal Event: Represents the reception of an asynchronous message or a named signal object from another object. It is a one-way communication where the sender does not wait for a reply.**

2. **Call Event: Represents the synchronous invocation of an operation on the object. When a call event is received, the object executes the specified operation, and the sender waits for the operation to complete.**

3. **Time Event: An event caused by the passage of a specific amount of time after entering a state or by a specific date/time being reached. It is specified using the keyword after (e.g., after(10 seconds)).**

4. **Change Event: An event that occurs when a specific condition or boolean expression becomes true. It is specified using the keyword when (e.g., when(temperature > 100)).**

Q4) b) What do you mean by Association ends? Explain N-ary association in short. [5]
Association Ends An association end is an endpoint of an association relationship where it connects to a class. Each end has properties that describe the role the class plays in that relationship. The key properties of an association end are:

- **Role Name: A name that clarifies the role of the class in the relationship. For example, in a relationship between a Person class and a Company class, the roles could be employee and employer.**

- **Multiplicity: Specifies how many objects of the class can participate in one instance of the relationship. It is expressed as a number or a range (e.g., 1, * for many, 0..1 for zero or one).**

- **Navigability: Indicates whether it's possible to traverse from one object to another across the association, shown as an arrow on the association line.**

N-ary Association An N-ary association is a relationship that connects three or more classes. While most associations are binary (connecting two classes), an n-ary association is used when a relationship is a single event or entity that intrinsically involves all participants. It is represented by a large diamond with lines connecting to all the participating classes.

- **Example: A single relationship could connect a Developer, a Project, and a ProgrammingLanguage to show that "a specific developer works on a specific project using a specific language." This single relationship cannot be broken down into binary associations without losing information.**

Q4) c) What is state? Explain with example transitions and condition. [5]

- **State: A state represents a condition in the life of an object during which it satisfies some property, performs some activity, or waits for an event. It is a snapshot of an object's condition at a point in time.**
  - **Example: A Door object can be in an Open state or a Closed state.**

- **Transition: A transition represents a change from one state (the source) to another (the target). It is triggered by an event and may involve an action. It is shown as an arrow in a state diagram.**
  - **Example: For the Door object, a transition from the Closed state to the Open state is triggered by the event openDoor. Closed -- openDoor --> Open**

- **Condition (Guard): A guard is a boolean expression that must evaluate to true for a transition to occur. It acts as a gatekeeper for the transition. If the event happens but the guard condition is false, the transition will not fire. It is written in square brackets [].**
  - **Example: A locked door cannot be opened. We can model this with a guard condition. Closed -- openDoor [isUnlocked] --> Open Here, the openDoor event will only trigger the transition from Closed to Open if the guard condition [isUnlocked] is true.**